

Nginx on EC2 Linux

Summary, Benefits, Advantages, and Command Explanation
And Deployment

Prepared by Mohit Gadilohar

Contents

1	Nginx Summary	1
1.	What happens when a user opens a website?	1
2	Why Is Nginx Used?	1
2.	Common uses of Nginx	1
3	Benefits of Nginx	1
1.	1. High performance	1
2.	2. Low resource usage	2
3.	3. Fast static-file delivery	2
4.	4. Reverse-proxy support	2
5.	5. Load balancing	2
6.	6. HTTPS and security features	2
7.	7. Simple service management	2
4	Advantages of Nginx	2
5	Command Explanation	2
1.	1. sudo su	2
2.	2. yum update -y	3
3.	3. yum install nginx -y	3
4.	4. cd /usr/share/nginx/html	4
5.	5. touch index.html	4
6.	6. vi index.html	4
6	Complete Command Sequence	5
7	Final Summary	5
8	References	6

1 Nginx Summary

Nginx, pronounced “engine-x,” is a high-performance web server. It receives requests from web browsers and returns website files such as HTML, CSS, JavaScript, images, and other resources. On an Amazon EC2 Linux instance, Nginx allows a website to be hosted and accessed through the instance’s public IP address or a domain name.

Nginx is also used as a reverse proxy, load balancer, and gateway for application servers. For a beginner web-server project, its main job is to serve a webpage from the server’s document root.

“Nginx is the software that listens for HTTP requests and delivers the requested website content to the browser.”

1. What happens when a user opens a website?

A browser sends an HTTP request to the server’s IP address. Nginx listens for that request, locates the requested file in its configured document root, and sends the file back as an HTTP response. When the requested path is /, Nginx commonly looks for a default file such as `index.html`.

Part	Explanation	Browser	Sends the website request	EC2 instance	Provides the virtual Linux server	Security group	Allows or blocks network traffic	Nginx	Handles the HTTP request	Document root	Stores files served by Nginx	<code>index.html</code>	Common default homepage
------	-------------	---------	---------------------------	--------------	-----------------------------------	----------------	----------------------------------	-------	--------------------------	---------------	------------------------------	-------------------------	-------------------------

2 Why Is Nginx Used?

Nginx is used because it can serve many static requests efficiently while using relatively few system resources. It is suitable for websites hosted on small cloud instances, including development and learning instances.

Nginx can serve static files directly. It can also forward requests to application servers such as Node.js, Python, Java, PHP-FPM, or other backend services. This separation allows Nginx to handle public web traffic while the application focuses on application logic.

1. Common uses of Nginx

Nginx is commonly used for the following purposes:

- Hosting static websites and HTML pages.
- Acting as a reverse proxy in front of an application.
- Distributing traffic across multiple backend servers.
- Terminating HTTPS/TLS connections.
- Caching selected responses.
- Serving images, stylesheets, scripts, and downloads.

3 Benefits of Nginx

1. 1. High performance

Nginx is designed to process many simultaneous connections efficiently. Its event-driven architecture allows it to handle requests without creating a separate heavy process for every connection.

2. 2. Low resource usage

Nginx can run effectively on a small EC2 instance. It generally uses less memory and CPU than a traditional process-per-connection design, although actual usage depends on traffic and configuration.

3. 3. Fast static-file delivery

Nginx is very effective at delivering files such as HTML, CSS, JavaScript, images, and videos. It reads the file from the document root and returns it to the client.

4. 4. Reverse-proxy support

Nginx can receive internet traffic and forward selected requests to an internal application server. The application server does not need to be exposed directly to the public internet.

5. 5. Load balancing

Nginx can distribute requests among multiple backend servers. This can improve availability and prevent one application server from receiving all traffic.

6. 6. HTTPS and security features

Nginx can manage TLS certificates, redirect HTTP to HTTPS, restrict access to selected locations, and add useful response headers. These features should be configured carefully for production use.

7. 7. Simple service management

On Amazon Linux, Nginx can be started, stopped, enabled at boot, and checked using `systemctl`. This makes the server easy to manage during deployment.

4 Advantages of Nginx

| Advantage | Meaning | |---| | Fast | Delivers static content quickly | | Scalable | Handles many concurrent connections efficiently | | Lightweight | Suitable for small and medium cloud instances | | Flexible | Works as a web server, reverse proxy, or load balancer | | Reliable | Can run continuously as a managed Linux service | | Popular | Has broad documentation and community support | | Cost-effective | The software is open source; AWS infrastructure costs still apply | | Extensible | Supports custom server blocks, caching, HTTPS, and proxy rules |

Nginx is not automatically the best choice for every workload. The appropriate web server depends on the application, team experience, modules required, operating system, and performance needs. Nginx is a strong general-purpose choice for static websites and as a front-end proxy.

5 Command Explanation

The following commands are the commands used in the basic EC2 Linux Nginx deployment. Connect to the EC2 instance through SSH before running them.

1. 1. `sudo su`

```
sudo su
```

This command opens a root shell. `sudo` means “superuser do,” so it runs the following command with administrative privileges. `su` means “switch user.” When no username is provided, `su` normally switches to the root user.

After this command, the prompt commonly changes from `$` to `#`. The root user can install software, change system files, and delete important data. Use root privileges carefully. A safer modern practice is to use `sudo` only for the individual command that needs it, but `sudo su` is often shown in beginner tutorials because it avoids repeating `sudo`.

To leave the root shell, run:

```
exit
```

2. `yum update -y`

```
yum update -y
```

This command updates installed Linux packages to newer available versions.

| Part | Meaning | |---| | `yum` | Package-management command used by many Linux systems | | `update` | Finds and installs available package updates | | `-y` | Automatically answers yes to confirmation prompts | Amazon Linux 2023 uses `dnf` internally, and `yum` is commonly available as a compatibility command or link. Therefore, both of the following forms may be seen:

```
yum update -y
```

Updating packages helps install security fixes and current dependencies before installing Nginx. A package update can change system files, so review the output in a production environment instead of accepting every change blindly.

3. `yum install nginx -y`

```
yum install nginx -y
```

This command downloads and installs the Nginx web server package and any required dependencies.

| Part | Meaning | |---| | `yum` | Package manager | | `install` | Installs a new software package | | `nginx` | Name of the web-server package | | `-y` | Automatically confirms installation prompts |

After installation, confirm that Nginx is available:

```
nginx -v
```

On Amazon Linux 2023, the equivalent command is commonly written as:

```
sudo install -y nginx
```

The two forms express the same installation goal when the `yum` compatibility command is available.

4. `cd /usr/share/nginx/html`

```
cd /usr/share/nginx/html
```

This command changes the current working directory to Nginx's default document root on Amazon Linux.

Part	Meaning
<code>cd</code>	Change directory
<code>/usr/share/nginx/html</code>	Directory containing the default website files

Nginx reads website files from its configured root directory. When a browser requests the root URL, Nginx commonly searches this directory for `index.html`.

Check the directory after entering it:

```
pwd
ls -la
```

`pwd` prints the current path. `ls -la` lists the files, ownership, and permissions.

5. `touch index.html`

```
touch index.html
```

This creates an empty file named `index.html` in the current directory. If the file already exists, `touch` normally updates its modification time without deleting its contents.

`index.html` is a common default homepage. Creating the file alone does not create visible webpage content; it only creates an empty file. HTML code must be added in the next step.

If the directory requires administrator permission, use:

```
sudo touch /usr/share/nginx/html/index.html
```

6. `vi index.html`

```
vi index.html
```

This opens `index.html` in the `vi` text editor. If the file is protected, use:

```
sudo vi /usr/share/nginx/html/index.html
```

`vi` has different modes. The most important beginner steps are:

Action	Keys
Open the file	<code>vi index.html</code>
Start typing	Press <code>i</code>
Stop typing	Press <code>Esc</code>
Save and exit	Type <code>:wq</code> , then press Enter
Exit without saving	Type <code>:q!</code> , then press Enter

After pressing `i`, type or paste a simple webpage such as:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>My Nginx Website</title>
```

```
</head>
<body>
  <h1>Hello from Nginx</h1>
  <p>This page is hosted on an EC2 Linux server.</p>
</body>
</html>
```

Press `Esc` after entering the HTML. Type `:wq` and press `Enter` to save the file and close `vi`.

6 Complete Command Sequence

The following sequence shows the commands in the same order as the basic deployment exercise:

```
sudo su
yum update -y
yum install nginx -y
cd /usr/share/nginx/html
touch index.html
vi index.html
```

After saving the HTML file, start Nginx with:

```
systemctl start nginx
systemctl enable nginx
systemctl status nginx
```

`systemctl start nginx` starts Nginx now. `systemctl enable nginx` configures it to start after a reboot.

`systemctl status nginx` shows whether the service is running.

Test the website locally:

```
curl http://localhost
```

Then open the EC2 instance's public IP address in a browser:

```
http://PUBLIC_IP_ADDRESS/
```

The EC2 security group must allow inbound HTTP traffic on port 80. Without that rule, Nginx may work locally while the browser cannot reach it from the internet.

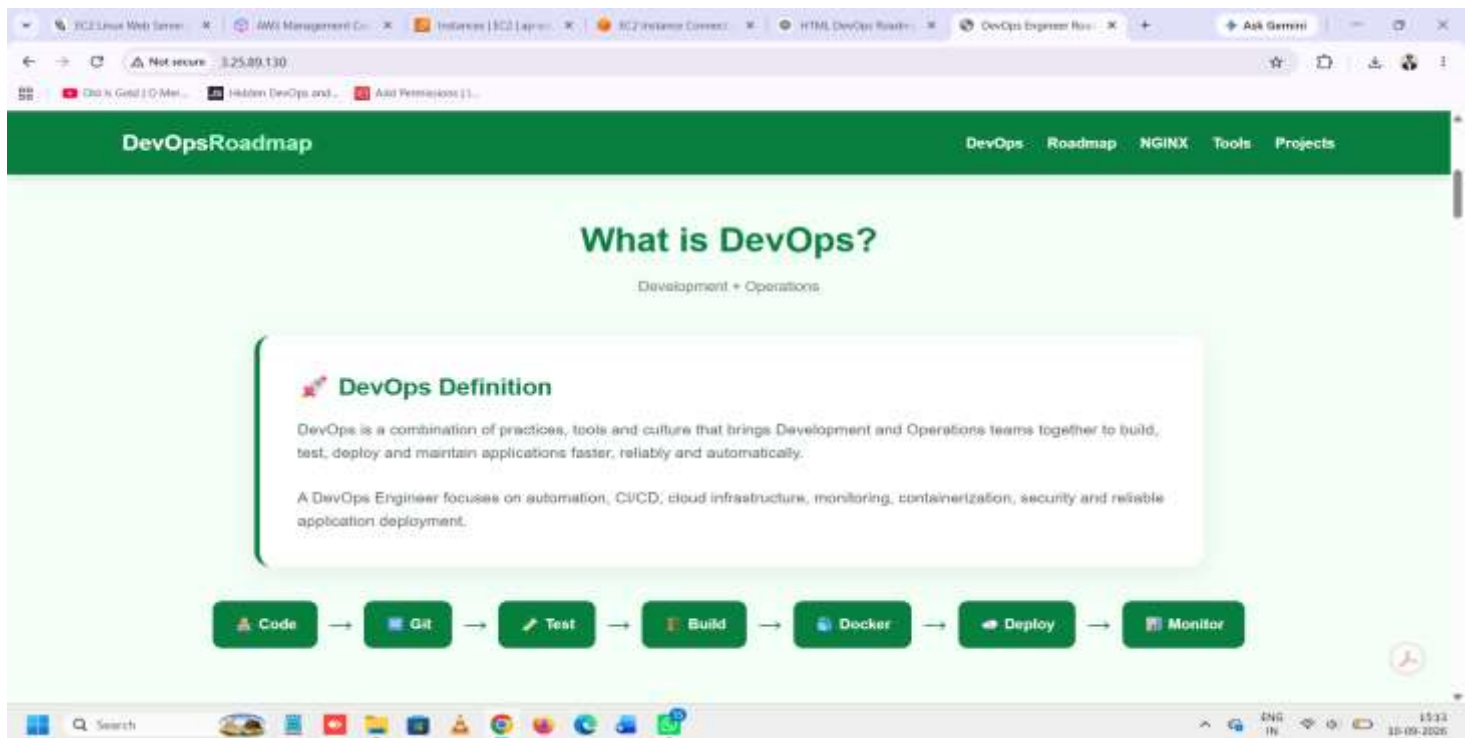
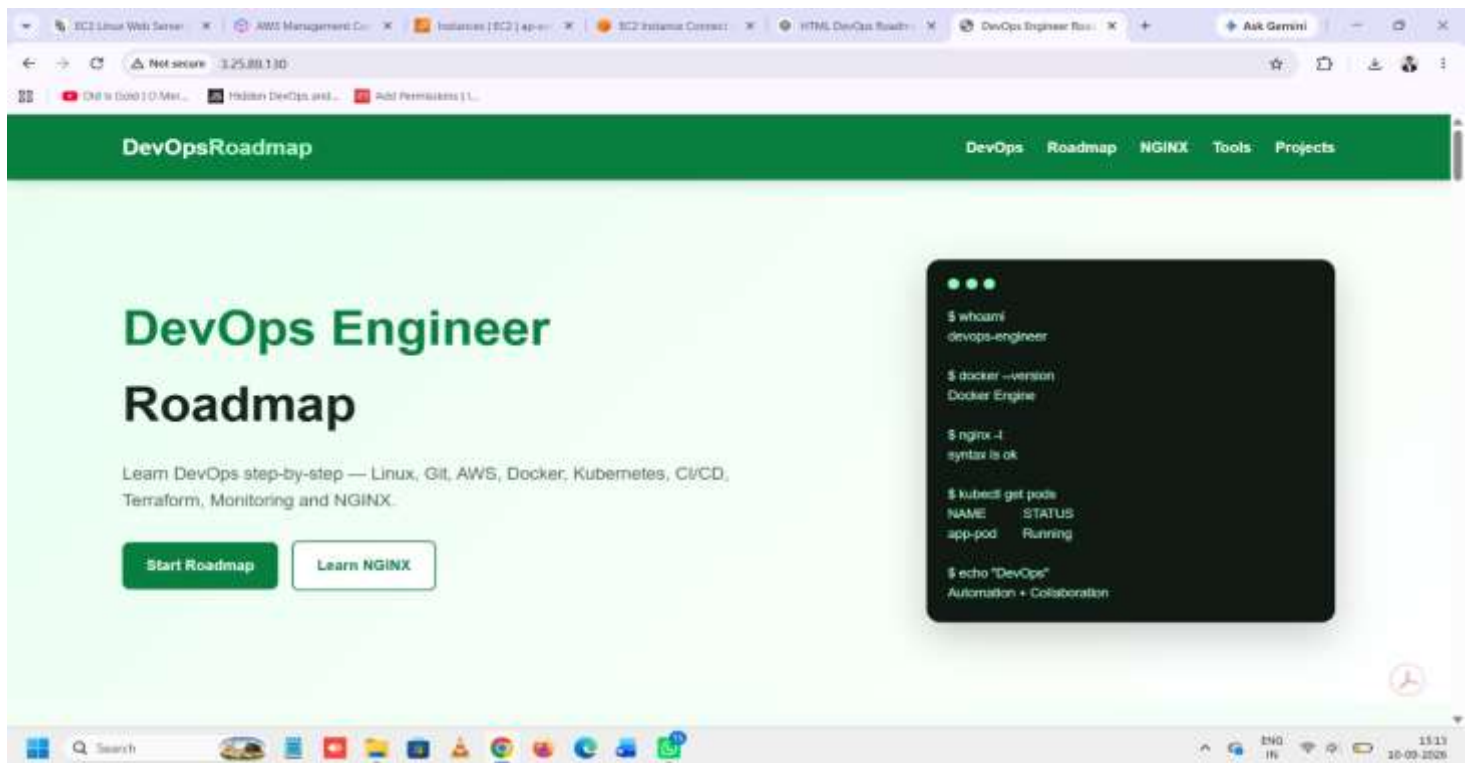
7 Final Summary

Nginx is a fast, lightweight, and flexible web server used to deliver website files and manage web traffic. It is popular on EC2 because it performs well on small instances, uses resources efficiently, supports static websites and reverse-proxy deployments, and provides useful features for HTTPS, caching, and load balancing.

The command sequence performs six main tasks: obtain administrative access, update the operating system, install Nginx, move to the document root, create the homepage file, and edit the homepage. After

the file is saved, Nginx must be started and the EC2 security group must allow port 80 for the webpage to be accessible.

8 Show Output



DevOpsRoadmap

DevOps Roadmap NGINX Tools Projects

NGINX for DevOps Engineers ⚡

High-performance web server, reverse proxy and load balancer.

What is NGINX?

NGINX is a high-performance web server that can also work as a reverse proxy, load balancer, HTTP cache and API gateway.

- Serve static websites
- Reverse proxy applications
- Load balancing
- SSL/TLS termination
- HTTP caching
- Application traffic management

NGINX Architecture



```
graph LR; User --> NGINX; NGINX --> AppServer[App Server]
```

NGINX receives client requests and forwards them to the appropriate backend application server.

⚙️ Important NGINX Commands

```
# Install NGINX sudo yum install nginx -y # Start NGINX sudo systemctl start nginx # Enable NGINX
sudo systemctl enable nginx # Check NGINX status sudo systemctl status nginx # Test configuration
sudo nginx -t # Restart NGINX sudo systemctl restart nginx # Stop NGINX sudo systemctl stop nginx
```

DevOpsRoadmap

DevOps Roadmap NGINX Tools Projects

DevOps Tools 🌸

Tools commonly used in a modern DevOps workflow.



Linux
Operating System



Git
Version Control



AWS
Cloud Platform



NGINX
Web Server



Docker
Containers



Jenkins
CI/CD



Kubernetes
Container Orchestration



Terraform
Infrastructure as Code

Browser tabs: ECE Linux Web Server, AWS Management Co..., Instances | EC2 | ne-sh..., EC2 Instance Connect, HTML DevOps Roadmap, DevOps Engineer Ro..., Ask Gemini

Address bar: Not secure 3.25.89.130

DevOpsRoadmap

DevOps Roadmap NGINX Tools Projects

DevOps Projects

Build projects to become job-ready

Project 01

EC2 Apache Web Server

Deploy an Apache HTTP web server on an AWS EC2 Linux instance and host a website.

Project 02

NGINX Reverse Proxy

Configure NGINX as a reverse proxy between users and backend application servers.

Project 03

Docker Application

Create a Dockerfile, build an image, run containers and expose application ports.

Project 04

CI/CD Pipeline

Build an automated pipeline that pulls code, tests, builds a Docker image and deploys it.

Project 05

AWS 3-Tier Architecture

Build a production-style architecture using Load Balancer, EC2, Auto Scaling and RDS.

Project 06

Monitoring & Alerts

Configure CloudWatch metrics and SNS notifications for infrastructure monitoring.

Windows taskbar: Search, icons for various applications, system tray showing 15:14 10-09-2026

Browser tabs: ECE Linux Web Server, AWS Management Co..., Instances | EC2 | ne-sh..., EC2 Instance Connect, HTML DevOps Roadmap, DevOps Engineer Ro..., Ask Gemini

Address bar: Not secure 3.25.89.130

DevOpsRoadmap

DevOps Roadmap NGINX Tools Projects

Job-Ready DevOps Path

Learn → Practice → Build Projects → Automate → Deploy → Monitor

Linux

 →

Git

 →

AWS

 →

NGINX

 →

Docker

 →

CI/CD

 →

Kubernetes

 →

Terraform

DevOps Engineer Roadmap

Learn Cloud • Automate Everything • Deploy Faster • Monitor Better

© 2026 DevOps Learning Portfolio

Windows taskbar: Search, icons for various applications, system tray showing 15:15 10-09-2026